

Quiz

In the computation model used in the lecture, how do we measure the running time of an algorithm? Select one answer.

- ☒ a. By the number of elementary operations it performs. (from lecture)
- b. By the number of digits in its output.
- c. By the number of seconds it takes on one particular laptop.

In Karatsuba's algorithm as presented in the lecture, the smaller multiplication problems are solved using the same idea again, until only single-digit multiplications remain.

☒ True = Recursion
False

What is the maximum subarray sum of $A = (4, -6, 3)$?

Answer:

$$1000n \leq O(n^2)$$

☒ True
False

$$\lim_{n \rightarrow \infty} \frac{1000n}{n^2} = \lim_{n \rightarrow \infty} \frac{1000}{n} = 0$$

anderer Lösungsweg. $1000n \leq 1000n^2 \leq O(n^2)$

Let $f(n) = 5n + 2$ for positive integers n . For each statement, decide whether it is true or false.

True	False		
	☒	$f \in \Theta(1)$.	
☒		$f \in \Theta(n)$.	
	☒	$f \in \Theta(n^2)$.	

$$\left. \begin{array}{l} 5n+2 \leq 5n+2n = 7n \leq O(n) \\ 5n+2 \geq 5n \geq \Omega(n) \end{array} \right\} \Rightarrow 5n+2 = \Theta(n)$$

For any real numbers x and y , if $x \leq 5$ and $y \leq 5$, then $x \leq y$.

True

False

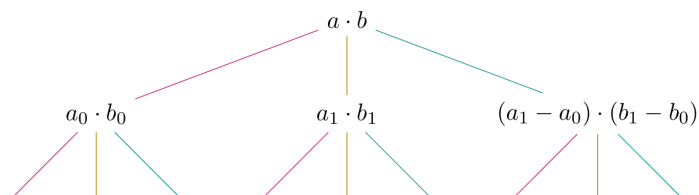
Gegenbeispiel: $x = 4, y = 2$

Karatsuba's algorithm multiplies two n -digit numbers, where we assume for simplicity that $n \geq 2$ is a power of two. How many recursive multiplication calls on two $n/2$ -digit numbers does one step make directly? Select one answer.

a. 2

☒ b. 3

c. 4



Let $S_k = \sum_{i=1}^k i^2$, where k is a positive integer. Which term must be added to S_k to obtain S_{k+1} ? Select one answer.

a. $k + 1$

☒ b. $(k + 1)^2$

c. k^2

d. i^2

e. $(i + 1)^2$

$$S_{k+1} = \sum_{i=1}^{k+1} i^2 = \underbrace{1^2 + 2^2 + 3^2 + \dots + (k-1)^2 + k^2}_{S_k} + (k+1)^2$$

For a list of $n \geq 3$ distinct integers, a recursive call has found the maximum a' of $a_1, \dots, a_{n-1}$. Which comparison is always sufficient to determine the maximum of the whole list $a_1, \dots, a_n$? Select one answer.

a. Compare a_1 with a_n .

b. Compare a_{n-1} with a_n .

☒ c. Compare a' with a_n .

$$[a_1, a_2, a_3, \dots, a_{n-2}, a_{n-1}, a_n]$$

$\underbrace{\hspace{10em}}_{\text{alle } \leq a'}$

(aus Exercise 22)

Let $A = (a_1, \dots, a_n)$ be an array of real numbers with $n \geq 5$. For each $j \in \{1, \dots, n\}$, let $S_j = \sum_{i=1}^j a_i$ be the sum of its first j entries. Which expression equals $a_3 + a_4 + a_5$? Select one answer.

☒ a. $S_5 - S_2$

b. $S_5 - S_3$

c. $S_5 + S_2$

$$[a_1, a_2, a_3, a_4, a_5, \dots]$$

$$S_2 = a_1 + a_2 \quad S_5 = a_1 + a_2 + a_3 + a_4 + a_5$$

$$S_5 - S_2 = a_3 + a_4 + a_5$$

What is the maximum subarray sum of $A = (-3, -1, -4)$?

Answer:

0

das leere Subarray mit Summe 0
ist immer auch eine mögliche Lösung

For any odd integer $b \in \mathbb{N}_0$, the following holds:

If $f : \mathbb{N}_0 \rightarrow \mathbb{N}_0$ satisfies $f(n+1) = b \cdot f(n)$ for all $n \in \mathbb{N}_0$, then $f(k)$ is odd for all $k \in \mathbb{N}_0$.

Here $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.

True

False

Gegenbeispiel: $f(0) = 0$ und $b = 1$

For any two functions $f, g : \mathbb{N}_0 \rightarrow \mathbb{N}_0$, the following holds:

If $f(0) > g(0)$ and $f(n+1) - f(n) > g(n+1) - g(n)$ for all $n \in \mathbb{N}_0$, then $f(k) > g(k)$ for all $k \in \mathbb{N}_0$.

Here $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.

True

False

Base Case: Sei $k=0$ $f(0) > g(0)$ stimmt, deshalb gilt die Aussage für $k=0$

Induktions-Hypothese: Wir nehmen an, dass die Aussage für ein beliebiges $k \in \mathbb{N}_0$ gilt.

$$\text{d.h. } f(k) > g(k)$$

Induktions-Schritt: Wir wollen die Aussage für $k+1$ zeigen, d.h. $f(k+1) > g(k+1)$

$$\begin{aligned} f(k+1) &= f(k+1) - f(k) + f(k) && \geq -f(k) + f(k) = 0 \\ &> g(k+1) - g(k) + \underbrace{f(k)}_{\text{I.H.}} && \begin{aligned} &\hookrightarrow \text{gegebene Eigenschaft.} \\ &f(k+1) - f(k) > g(k+1) - g(k) \end{aligned} \\ &> g(k+1) - g(k) + \underbrace{g(k)}_{\text{I.H.}} && \hookrightarrow -g(k) + g(k) = 0 \\ &= g(k+1) \end{aligned}$$

Somit ist die Aussage für alle $k \geq 0$ bewiesen per Induktion.

Discussion of Exercise Sheet 2

Exercise 2.1 Induction for inequalities.

(a) Prove via mathematical induction that for all integers $n \geq 5$,

$$2^n > n^2.$$

In your solution, you should address the base case, the induction hypothesis and the induction step.

Base Case: Sei $k=5$

$$2^5 = 32 > 25 = 5^2 \text{ stimmt, deshalb gilt die Aussage f\"ur } k=5$$

Induktions-Hypothese: Wir nehmen an, dass die Aussage f\"ur ein beliebiges $k \in \mathbb{N}$, $k \geq 5$ gilt.

$$\text{d.h. } 2^k > k^2$$

Induktions-Schritt: Wir wollen die Aussage f\"ur $k+1$ zeigen, d.h. $2^{k+1} > (k+1)^2$

$$2^{k+1} = 2 \cdot 2^k$$

$$\stackrel{\text{IH}}{>} 2 \cdot k^2$$

$$= k^2 + k^2$$

$$\hookrightarrow k^2 = k \cdot k \geq 5k \text{ weil } k \geq 5$$

$$\geq k^2 + 5k$$

$$= k^2 + 2k + 3k$$

$$\hookrightarrow 3k \geq 15 > 1 \text{ weil } k \geq 5$$

$$> k^2 + 2k + 1$$

$$\hookrightarrow \text{binomische Formel}$$

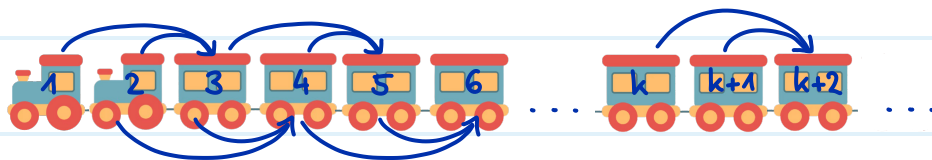
$$= (k+1)^2$$

Somit ist die Aussage f\"ur alle $n \geq 5$ bewiesen per Induktion.

(b) Recall that the Fibonacci numbers are defined by $f_0 = 0$, $f_1 = 1$ and the recursion relation $f_{n+1} = f_n + f_{n-1}$ for all $n \geq 1$. For example, $f_2 = 1$, $f_5 = 5$, $f_{10} = 55$, $f_{15} = 610$.

Prove via mathematical induction that $f_n \geq \frac{1}{3} \cdot 1.5^n$ for $n \geq 1$.

In your solution, you should address the base case, the induction hypothesis and the induction step.



Base Case: Sei $k=1$. $f_1 = 1 \geq 0.5 = \frac{1}{3} \cdot 1.5^1$ stimmt, deshalb gilt die Aussage für $k=1$

Sei $k=2$. $f_2 = 1 \geq 0.75 = \frac{1}{3} \cdot 1.5^2$ stimmt, deshalb gilt die Aussage für $k=2$

Induktions-Hypothese: Wir nehmen an, dass die Aussage für ein beliebiges $k \in \mathbb{N}$ und $k+1$ gilt,

$$\text{d.h. } f_k \geq \frac{1}{3} \cdot 1.5^k \text{ und } f_{k+1} \geq \frac{1}{3} \cdot 1.5^{k+1}$$

Induktions-Schritt: Wir wollen die Aussage für $k+2$ zeigen, d.h. $f_{k+2} \geq \frac{1}{3} \cdot 1.5^{k+2}$

$$\begin{aligned} f_{k+2} &\stackrel{\text{def}}{=} f_{k+1} + f_k \\ &\stackrel{\text{IH}}{\geq} \frac{1}{3} \cdot 1.5^{k+1} + \frac{1}{3} \cdot 1.5^k && \text{ausklammern} \\ &= \frac{1}{3} \cdot 1.5^k (1.5 + 1) \\ &= \frac{1}{3} \cdot 1.5^k \cdot 2.5 && 1.5^2 = 2.25 < 2.5 \\ &> \frac{1}{3} \cdot 1.5^k \cdot 1.5^2 && \text{Potenzgesetz} \\ &= \frac{1}{3} \cdot 1.5^{k+2} \end{aligned}$$

Somit ist die Aussage für alle $n \geq 1$ bewiesen per Induktion.

Exercise 2.2 Induction for algorithms (2 bonus points).

We are given a list $a_1, \dots, a_n \in \mathbb{N}$ of $n \geq 1$ distinct integers and our task is to find the maximum element a^* in this list:

- (i) $a^* \in \{a_1, \dots, a_n\}$.
- (ii) $a^* \geq a_i$ for all $i = 1, \dots, n$.

Consider the following recursive algorithm:

Algorithm 1 Recursive maximum

```

if  $n = 1$  then
  return  $a_1$ 
else
  Apply this algorithm recursively and find the maximum element  $a'$  among  $a_1, \dots, a_{n-1}$ 
  if  $a' > a_n$  then
    return  $a'$ 
  else
    return  $a_n$ 

```

$\triangleright n \geq 2$
 $\triangleright a' \leq a_n$

Prove via mathematical induction that this algorithm correctly returns the maximum element of the given list of numbers.

Aussage: Der Algorithmus gibt das Maximum der Liste korrekt zurück

Base Case: Sei $k=1$ Der Algorithmus gibt a_1 zurück, welches das einzige Element der Liste ist. Es gilt $a_1 \in \{a_1\}$ und $a_1 \geq a_1$, d.h. (1) und (2) sind erfüllt. Deshalb gilt die Aussage für $k=1$.

Induktions-Hypothese: Wir nehmen an, dass die Aussage für ein beliebiges $k \in \mathbb{N}$ gilt.

d.h. "Der Algorithmus gibt das Maximum der Liste korrekt zurück für Länge k "

Induktions-Schritt: Wir wollen die Aussage für $k+1$ zeigen, d.h. "Der Algorithmus gibt das Maximum der Liste korrekt zurück für Länge $k+1$ "

Da $k+1 > 1$ ist, wird der Algorithmus rekursiv mit Länge k ausgeführt

Die I.H. garantiert, dass $a' \in \{a_1, \dots, a_k\}$ und $a' \geq a_i$ für $i=1, \dots, k$

Case $a' > a_{k+1}$: Der Algorithmus gibt a' zurück

Bedingung (1) ist erfüllt, da $a' \in \{a_1, \dots, a_k\}$ per I.H

und somit gilt auch $a' \in \{a_1, \dots, a_{k+1}\}$

Bedingung (2) ist erfüllt, da $a' \geq a_i$ für $i=1, \dots, k$ per I.H

und somit gilt zusammen mit $a' > a_{k+1}$ auch $a' \geq a_i$ für $i=1, \dots, k+1$

Case $a' \leq a_{k+1}$: Der Algorithmus gibt a_{k+1} zurück

Bedingung (1) ist erfüllt, da $a_{k+1} \in \{a_1, \dots, a_{k+1}\}$

Bedingung (2) ist erfüllt, da $a' \geq a_i$ für $i=1, \dots, k$ per I.H

Zusammen mit $a_{k+1} \geq a'$ gilt: $a_{k+1} \geq a' \geq a_i$ für $i=1, \dots, k$

und somit $a_{k+1} \geq a_i$ für $i=1, \dots, k+1$

Somit ist die Aussage für alle $n \geq 1$ bewiesen per Induktion.

(a) For all the following functions the variable n ranges over $\mathbb{N}$. Prove or disprove the following statements. Justify your answer using Theorems 1 and/or 2.

(1) $2n^5 + 10n^2 \leq O(\frac{1}{100}n^6)$

$$\lim_{n \rightarrow \infty} \frac{2n^5 + 10n^2}{\frac{1}{100} n^6} = \lim_{n \rightarrow \infty} \frac{2n^5}{\frac{1}{100} n^6} + \frac{10n^2}{\frac{1}{100} n^6} = \lim_{n \rightarrow \infty} \underbrace{\frac{2}{\frac{1}{100} n}}_{\rightarrow 0} + \underbrace{\frac{10}{\frac{1}{100} n^4}}_{\rightarrow 0} = 0 \Rightarrow \text{True}$$

$$(2) \quad n^{10} + 2n^2 + 7 \leq O(100n^9)$$

$$\lim_{n \rightarrow \infty} \frac{n^{10} + 2n^2 + 7}{100n^9} = \lim_{n \rightarrow \infty} \frac{n^{10}}{100n^9} + \frac{2n^2}{100n^9} + \frac{7}{100n^9} = \lim_{n \rightarrow \infty} \underbrace{\frac{n}{100}}_{\rightarrow \infty} + \underbrace{\frac{2}{100n^7}}_{\rightarrow 0} + \underbrace{\frac{7}{100n^9}}_{\rightarrow 0} = \infty \Rightarrow \text{False}$$

$$(3) \quad e^{1.2n} \leq O(e^n)$$

$$\lim_{n \rightarrow \infty} \frac{e^{1.2n}}{e^n} \underset{\substack{\uparrow \\ \text{Potenzgesetz}}}{=} \lim_{n \rightarrow \infty} e^{1.2n - n} = \lim_{n \rightarrow \infty} e^{0.2n} = \infty \Rightarrow \text{False}$$

(4)* $n^{\frac{2n+3}{n+1}} \leq O(n^2)$ (This question does not count toward the bonus points.)

$$\lim_{n \rightarrow \infty} \frac{n^{\frac{2n+3}{n+1}}}{n^2} = \lim_{n \rightarrow \infty} n^{\frac{2n+3}{n+1} - 2} = \lim_{n \rightarrow \infty} n^{\frac{2n+3}{n+1} - \frac{2(n+1)}{n+1}}$$

↪ Potenzgesetz
↪ er itern

$$= \lim_{n \rightarrow \infty} n^{\frac{2n+3-2(n+1)}{n+1}} = \lim_{n \rightarrow \infty} n^{\frac{1}{n+1}} = \lim_{n \rightarrow \infty} e^{\ln(n^{\frac{1}{n+1}})} = \lim_{n \rightarrow \infty} e^{\underbrace{\frac{1}{n+1} \ln(n)}_{\rightarrow 0}} = 1$$

auf einen Nenner bringen $e^{\ln}$ Trick log. Gesetz müssen wir noch zeigen

$$\lim_{n \rightarrow \infty} \frac{\ln(n)}{n+1} \stackrel{\text{L'Hôp}}{=} \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{1} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0 \quad \Rightarrow \underline{\underline{\text{True}}}$$

Exercise 2.5 Asymptotic growth of $\ln(n!)$.

Recall that the factorial of a positive integer n is defined as $n! = 1 \cdot 2 \cdot \dots \cdot (n-1) \cdot n$. For the following functions n ranges over $\mathbb{N}_{\geq 2}$.

(a) Show that $\ln(n!) \leq O(n \ln n)$.

Hint: You can use the fact that $n! \leq n^n$ for $n \in \mathbb{N}_{\geq 2}$ without proof.

$$\ln(n!) \leq \ln(n^n) = n \ln n \leq O(n \ln n)$$

$\swarrow$
 log. Gesetz

(b) Show that $n \ln n \leq O(\ln(n!))$.

Hint: You can use the fact that $\left(\frac{n}{2}\right)^{\frac{n}{2}} \leq n!$ for $n \in \mathbb{N}_{\geq 2}$ without proof.

$$\begin{aligned}
 \lim_{n \rightarrow \infty} \frac{n \ln n}{\ln(n!)} &\leq \lim_{n \rightarrow \infty} \frac{n \ln n}{\ln\left(\left(\frac{n}{2}\right)^{\frac{n}{2}}\right)} && \swarrow \text{log. Gesetz} \\
 &= \lim_{n \rightarrow \infty} \frac{n \ln n}{\frac{n}{2} \ln\left(\frac{n}{2}\right)} && \swarrow \text{kürzen, und 2 nach oben nehmen} \\
 &= \lim_{n \rightarrow \infty} \frac{2 \ln n}{\ln\left(\frac{n}{2}\right)} && \swarrow \text{log. Gesetz} \\
 &= \lim_{n \rightarrow \infty} \frac{2 \ln n}{\ln n - \ln 2} \\
 &= \lim_{n \rightarrow \infty} \frac{2}{1 - \underbrace{\frac{\ln 2}{\ln n}}_{\rightarrow 0}} && \swarrow \text{ln kürzen} \\
 &= 2
 \end{aligned}$$

Maximum Subarray Sum

Gegeben: Array der Länge n . Schreibweise $A[1, \dots, n]$ oder $(a_1, a_2, \dots, a_n)$

Gesucht: Startindex i und Endindex j , wobei $1 \leq i \leq j \leq n$,

sodass $\sum_{k=i}^j a_k$ maximal ist. (Die leere Summe ist zusätzlich auch erlaubt)

Beispiel: $[2, -5, 4, 1, -2, 7, -3, -14, 1, 2]$

$$\sum_{k=3}^6 a_k = 4 + 1 - 2 + 7 = \underline{\underline{10}}$$

Teilproblem: Randmaximum $R_i :=$ grösste Summe eines Subarrays, das in Index i endet
(also i inklusiv)

Beispiel: Array: $[2, -5, 4, 1, -2, 7, -3, -14, 1, 2]$

Randmaximum: $[2, -3, 4, 5, 3, 10, 7, -7, 1, 3]$
* * * (Subarray neu beginnen)

Lösung: $\max_{1 \leq i \leq n} R_i$ oder 0 falls alle Arrayelemente negativ sind

MSS-INDUKTIV($a_1, \dots, a_n$) Laufzeit: $\Theta(n)$

```

1 randmax  $\leftarrow$  0
2 maxS  $\leftarrow$  0
3 Für  $i \leftarrow 1, \dots, n$ :
4   randmax  $\leftarrow$  randmax +  $a_i$ 
5   Wenn randmax > maxS:
6     maxS  $\leftarrow$  randmax
7   Wenn randmax < 0:
8     randmax  $\leftarrow$  0
9 Gib maxS zurück.
```

- langsamere Algorithmen für MSS sind weniger wichtig. Sie wurden vor allem gezeigt, um euch die algorithmische Denkweise näher zu bringen

Θ und Ω - Notation

Intuitiv

$O(f)$:= Menge aller Funktionen, die asymptotisch langsamer oder gleich schnell wachsen

$\Theta(f)$:= Menge aller Funktionen, die asymptotisch gleich schnell wachsen

$\Omega(f)$:= Menge aller Funktionen, die asymptotisch gleich schnell oder schneller wachsen

Mathematisch

$$O(f) := \{ g: \mathbb{N} \rightarrow \mathbb{R}^+ \mid \exists C > 0 \forall n \in \mathbb{N} \ g(n) \leq C \cdot f(n) \}$$

$$\Theta(f) := \{ g: \mathbb{N} \rightarrow \mathbb{R}^+ \mid f \leq O(g) \text{ und } g \leq O(f) \} = O(f) \cap \Omega(f)$$

$$\Omega(f) := \{ g: \mathbb{N} \rightarrow \mathbb{R}^+ \mid f \leq O(g) \}$$

We can restate Theorem 1 from exercise sheet 2 as follows.

Theorem 1. Let N be an infinite subset of $\mathbb{N}$ and $f: N \rightarrow \mathbb{R}^+$ and $g: N \rightarrow \mathbb{R}^+$.

- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, then $f \leq O(g)$, but $f \neq \Theta(g)$. g wächst schneller (oder gleich schnell)
- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = C \in \mathbb{R}^+$, then $f = \Theta(g)$.
and $g = \Theta(f)$ beide wachsen gleich schnell
- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$, then $f \geq \Omega(g)$, but $f \neq \Theta(g)$. f wächst schneller (oder gleich schnell)

• $g = \Theta(f)$ kann man alternativ auch zeigen, indem man $f \leq O(g)$ und $g \leq O(f)$

oder $f \leq O(g)$ und $f \geq \Omega(g)$

oder $g \leq O(f)$ und $g \geq \Omega(f)$

oder $f \geq \Omega(g)$ und $g \geq \Omega(f)$ zeigt.

Aufgaben:

True or False: $n + \log n = \Theta(n)$

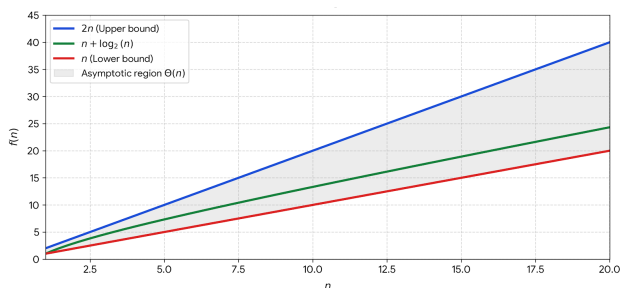
obere Schranke: $n + \log n \leq n + n = 2n \leq O(n)$

untere Schranke: $n + \log n \geq n \geq \Omega(n)$

daraus folgt: $n + \log n = \Theta(n)$

anderer Lösungsweg: $\lim_{n \rightarrow \infty} \frac{n + \log n}{n} = \lim_{n \rightarrow \infty} \frac{n + \frac{\ln n}{\ln 2}}{n} \stackrel{\text{L'H\^op}}{=} \lim_{n \rightarrow \infty} \frac{1 + \frac{\frac{1}{n}}{\ln 2}}{1} = \lim_{n \rightarrow \infty} 1 + \underbrace{\frac{1}{n \cdot \ln 2}}_{\rightarrow 0} = 1$

↖
Basiswechsel



True or False: $\log(n^5 + 3n) = \Theta(\log n)$

obere Schranke: $\log(n^5 + 3n) \leq \log(n^5 + 3n^5) = \log(4n^5) = \log(4) + \log(n^5)$
 $\stackrel{\log \text{ Gesetz}}{\sim} \log(4) + 5 \log(n) \stackrel{\log \text{ Gesetz}}{\sim} 5 \log(n) \leq O(\log n)$

untere Schranke: $\log(n^5 + 3n) \geq \log(n^5) \stackrel{\log \text{ Gesetz}}{\sim} 5 \log(n) \geq \Omega(\log n)$

daraus folgt: $\log(n^5 + 3n) = \Theta(\log n)$

True or False: $2^n \geq \Omega(2^{n+3})$

$$\lim_{n \rightarrow \infty} \frac{2^n}{2^{n+3}} = \lim_{n \rightarrow \infty} \frac{2^n}{2^n 2^3} = \lim_{n \rightarrow \infty} \frac{1}{2^3} = \frac{1}{8}$$

True or False: $\sqrt{n^2 + 7n + 1} = \Theta(n)$

obere Schranke: $\sqrt{n^2 + 7n + 1} \leq \sqrt{n^2 + 7n^2 + n^2} = \sqrt{9n^2} = 3n \leq O(n)$

untere Schranke: $\sqrt{n^2 + 7n + 1} \geq \sqrt{n^2} = n \geq \Omega(n)$

daraus folgt: $\sqrt{n^2 + 7n + 1} = \Theta(n)$

True or False: $\sin(n^3) + n \geq \Omega(n^2)$

$$\lim_{n \rightarrow \infty} \frac{\sin(n^3) + n}{n^2} \leq \lim_{n \rightarrow \infty} \frac{1 + n}{n^2} = \lim_{n \rightarrow \infty} \underbrace{\frac{1}{n^2}}_{\rightarrow 0} + \underbrace{\frac{n}{n^2}}_{\rightarrow 0} = \lim_{n \rightarrow \infty} \frac{1}{n^2} + \frac{1}{n} = 0$$

weil $\sin(x) \leq 1$
für alle $x \in \mathbb{R}$

$$\lim_{n \rightarrow \infty} \frac{\sin(n^3) + n}{n^2} \geq \lim_{n \rightarrow \infty} \frac{-1 + n}{n^2} = \lim_{n \rightarrow \infty} \underbrace{-\frac{1}{n^2}}_{\rightarrow 0} + \underbrace{\frac{n}{n^2}}_{\rightarrow 0} = \lim_{n \rightarrow \infty} -\frac{1}{n^2} + \frac{1}{n} = 0$$

weil $\sin(x) \geq -1$
für alle $x \in \mathbb{R}$

Da der Limes ≤ 0 und ≥ 0 ist, muss er $= 0$ sein.

Dies zeigt, dass $\sin(n^3) + n \not\geq \Omega(n^2)$

(Manchmal "Sandwich Theorem" genannt)

Counting function calls

Algorithm 1

```

i ← 0
while i ≤ 3n do
    f()
    i ← i + 1

j ← 1
while j ≤ n + 5 do
    g()
    g()
    j ← j + 1
  
```

$$\begin{aligned}
 \text{Number of calls} &= \sum_{i=0}^{3n} 1 + \sum_{j=1}^{n+5} 2 \\
 &\quad \downarrow \text{+1 weil es bei 0 startet} \\
 &= (3n+1) \cdot 1 + (n+5) \cdot 2 \\
 &= 3n+1 + 2n+10 \\
 &= 5n+11 \\
 &= \Theta(n)
 \end{aligned}$$

Algorithm 2

```

i ← 1
while i ≤ n do
    j ← 1
    while j ≤ n do
        f()
        j ← j + 1
    i ← i + 1

k ← 0
while k ≤ 2n do
    g()
    k ← k + 1
  
```

$$\begin{aligned}
 \text{Number of calls} &= \sum_{i=1}^n \sum_{j=1}^n 1 + \sum_{k=0}^{2n} 1 \\
 &\quad \downarrow \text{+1 weil es bei 0 startet} \\
 &= \sum_{i=1}^n n + (2n+1) \cdot 1 \\
 &= n^2 + 2n + 1 \\
 &= \Theta(n^2)
 \end{aligned}$$

assume $n = k^2$ for some $k \in \mathbb{N}$

Algorithm 3 (more difficult)

$i \leftarrow 1$

while $i \leq n^2$ **do**

$t \leftarrow 1$

while $t \leq i$ **do**

$f()$

$t \leftarrow t + 1$

$i \leftarrow i + 1$

$j \leftarrow 0$

while $j^2 \leq n$ **do**

$g()$

$j \leftarrow j + 1$

$j^2 \leq n \Leftrightarrow j \leq \sqrt{n}$

$$\begin{aligned}
 \text{Number of calls} &= \sum_{i=1}^{n^2} \sum_{t=1}^i 1 + \sum_{j=0}^{\lfloor \sqrt{n} \rfloor} 1 \\
 &= \sum_{i=1}^{n^2} i + (\sqrt{n} + 1) \cdot 1 \\
 &\quad \text{Gauss'sche Summenformel} \quad \swarrow \text{+1 weil es bei 0 startet} \\
 &= \frac{n^2(n^2+1)}{2} + \sqrt{n} + 1 \\
 &= \frac{1}{2}n^4 + \frac{1}{2}n^2 + \sqrt{n} + 1 \\
 &= \Theta(n^4)
 \end{aligned}$$

Beweismethode für Summen

Assume that n is even

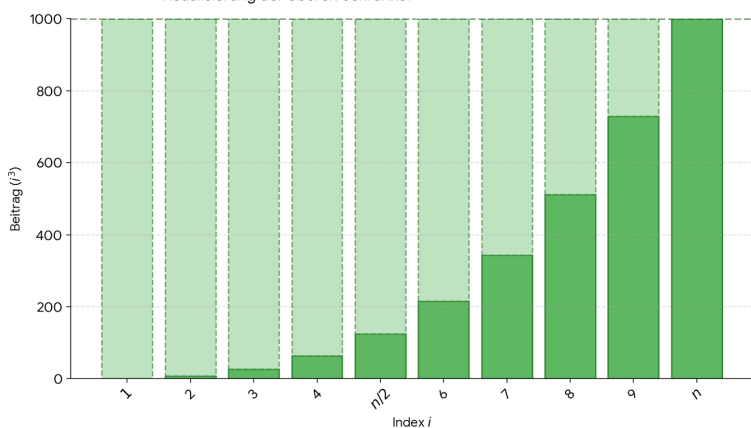
Show that $\sum_{i=1}^n i^3 = \Theta(n^4)$

obere Schranke: $\sum_{i=1}^n i^3 \leq \sum_{i=1}^n n^3 = n n^3 = n^4 \leq O(n^4)$

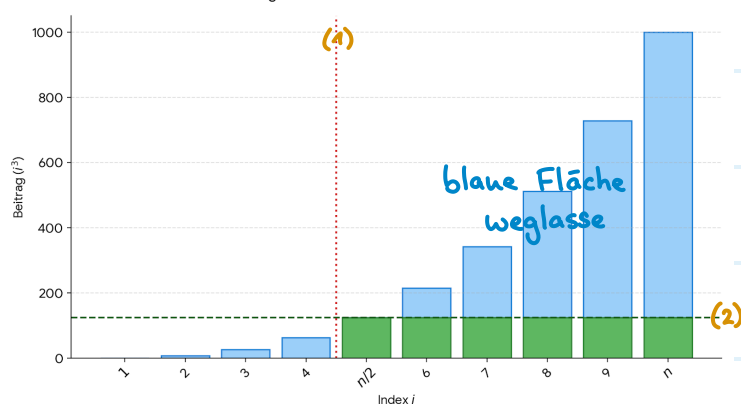
untere Schranke: $\sum_{i=1}^n i^3 \stackrel{(1)}{\geq} \sum_{i=\frac{n}{2}}^n i^3 \stackrel{(2)}{\geq} \sum_{i=\frac{n}{2}}^n \left(\frac{n}{2}\right)^3 \geq \left(\frac{n}{2}\right) \left(\frac{n}{2}\right)^3 = \left(\frac{n}{2}\right)^4 = \frac{1}{16} n^4 \geq \Omega(n^4)$

daraus folgt: $\sum_{i=1}^n i^3 = \Theta(n^4)$

Visualisierung der oberen Schranke:



Visualisierung der unteren Schranke:



Assume that n is even

Show that $\sum_{i=1}^n \sqrt{i} = \Theta(n^{1.5})$

obere Schranke: $\sum_{i=1}^n \sqrt{i} \leq \sum_{i=1}^n \sqrt{n} = n \sqrt{n} = n^{1.5} \leq O(n^{1.5})$

untere Schranke: $\sum_{i=1}^n \sqrt{i} \geq \sum_{i=\frac{n}{2}}^n \sqrt{i} \geq \sum_{i=\frac{n}{2}}^n \sqrt{\frac{n}{2}} \geq \frac{n}{2} \sqrt{\frac{n}{2}} = \left(\frac{n}{2}\right)^{1.5} = \frac{1}{2^{1.5}} n^{1.5} \geq \Omega(n^{1.5})$

daraus folgt: $\sum_{i=1}^n \sqrt{i} = \Theta(n^{1.5})$